Clock-Assisted Consensus in OmniPaxos-KV with Nezha-style Deadline-Ordered Multicast

Lorenzo Deflorian, Riccardo Fragale, Jouzas Skarbalius

March 16, 2026

1 Introduction

Large-scale distributed systems increasingly rely on consensus protocols to provide strongly consistent services over unreliable networks. Classical leader-based protocols such as Paxos and Raft guarantee linearizability but often pay a performance cost: every client operation typically requires at least two round trips to the leader, and network reordering or failures can further increase latency. Recent work on clock-assisted consensus, in particular the Nezha protocol, demonstrates that when nodes share tightly synchronized clocks, it becomes possible to speculatively process requests in deadline order and obtain a fast path that completes in effectively one round trip.

This project explores the hypothesis that *better clock synchronization leads to better consensus performance*. Building on the OmniPaxos-KV codebase, we integrate a configurable clock simulator and a Nezha-inspired, deadline-ordered multicast (DOM) layer into the existing key-value store. Our design uses synchronized logical clocks to attach deadlines to client requests and enforces a global ordering based on those deadlines, while preserving linearizability even when clock quality is poor or the network behaves adversarially.

Concretely, we extend OmniPaxos-KV with three main components. First, we implement a pluggable clock simulator that models drift, synchronization frequency, and uncertainty bounds, and exposes a simple API to the rest of the system. Second, we introduce a DOM-like layer that buffers and releases requests in deadline order, providing an optimistic fast path when requests arrive on time. Third, we add a proxy and we modify the server logic to support a fast path, where a quorum of replicas speculatively execute and reply early, and a slow path, where the system safely falls back to normal OmniPaxos log replication whenever deadlines are missed or ambiguity arises.

We evaluate our design using a suite of benchmarks that vary clock quality, deadline strategy, and failure conditions. All experiments are run with a fixed deployment of three servers, two clients, and one proxy. Our results show that tighter clock synchronization increase the fraction of operations that complete on the fast path, thereby improving both latency and throughput, while the

system remains linearizable under clock skew and node failures by transparently switching to the slow path when necessary.

The code for this project is available at `https://github.com/TheArtemis/omnipaxos-kv`.

2 Design

Our design adds a Nezha-style, clock-assisted fast path on top of the existing OmniPaxos-KV architecture. The key idea is to use synchronized clocks to tag each client request with a deadline and to ensure that servers process requests in increasing deadline order whenever they arrive on time. When this optimistic assumption fails, the system reverts to a conservative slow path that is backed by standard OmniPaxos log replication.

2.1 Clock Simulator

The foundation of our design is a configurable clock simulator, exposed through the `ClockSim` interface. Each node maintains its own simulated clock, parameterized by a drift rate, an uncertainty bound, and a synchronization frequency. The drift rate controls how quickly the simulated time deviates from real time, while the synchronization frequency determines how often the clock is "pulled" back towards a reference time. The uncertainty bound represents the maximum deviation that upper layers must assume when reasoning about deadlines.

The simulator provides two main methods, `get_time()` and `get_uncertainty()`, which return the current simulated time (in microseconds) and the current uncertainty estimate, respectively. All components that need time, notably the proxy and the DOM layer on the servers, depend exclusively on this interface rather than on the system clock. This lets us experiment with different clock qualities and resynchronization intervals without changing the rest of the code.

2.2 DOM-style Deadline-Ordered Processing

On top of the clock simulator, we implement a Nezha-inspired, DOM-like message processing layer. The proxy assigns each incoming client request a send time and a deadline, and forwards the request as a DOM message to all servers. Each server maintains two logical buffers:

- **Early buffer**: a priority queue ordered by deadline (and implicitly by client and request identifiers). Requests that arrive before their deadlines are inserted into this buffer. The DOM layer periodically checks the earliest pending deadline and releases all requests whose deadlines have been reached, preserving increasing deadline order.

- **Late buffer**: a secondary store for requests that cannot be safely inserted into the early buffer, for example because they arrive after their deadline or because they would violate the monotonicity of deadlines. These requests

are not processed on the fast path but may still be committed later via the slow path.

The server event loop queries the DOM layer for the duration until the next deadline and sleeps accordingly. When a deadline expires, the DOM layer returns a batch of *committable* messages from the early buffer. This provides a deterministic, deadline-based order that is consistent across replicas as long as messages arrive on time and within the uncertainty window.

2.3 Proxy-Centered Fast and Slow Paths

The proxy is responsible for interfacing with clients and orchestrating fast and slow paths. When a client sends a write or read request, the proxy records the current simulated time and computes a deadline using either a fixed default value or an adaptive value learned from previous runs (Section 2.5). From this, it also derives a fast-path timeout that bounds how long it is willing to wait for fast replies from the servers.

The proxy then multicasts the DOM-annotated request to all servers. As fast replies arrive, the proxy groups them per client request and tracks ballot, replica identity, and log-hash consistency. If a super-quorum of consistent fast replies is gathered before the fast-path timeout and a leader reply with a concrete result is present, the proxy concludes that the request has been safely processed in deadline order and replies to the client with the result. If this committable condition is not reached before timeout, the proxy aborts the fast path for that request and emits a fast-path abort message to all servers. This triggers a fallback where the request is ordered and committed through the OmniPaxos log and the client receives a slow-path reply once the decision is durable.

2.4 Leader Intervention

Leader intervention is required whenever a request cannot safely continue on the optimistic deadline-ordered path. In our implementation it happens in different situations. First of all, when the message immediately goes to late buffer the leader starts OmniPaxos for that request. Secondly, in case of fast path aborts the leader is forced to start OmniPaxosalso in this case so it's actively intervening. Finally the leader has to force the execution of all the followers also when some messages contain overlapping uncertainties. Concretely, `handle_overlapping_uncertainty()` classifies such ambiguous requests as non-committable for the early path and moves them to the late buffer.

Once requests are in the late buffer, the current leader is responsible for converting them into durable order: during `drain_late_buffer()`, it pops late-buffer entries and appends them to OmniPaxos.

2.5 Adaptive Deadlines and One-Way Delay Estimation

Static deadlines must trade off between two conflicting goals: they must be long enough to tolerate network jitter and clock uncertainty but short enough

to avoid adding unnecessary waiting time. To better navigate this trade-off, our design includes an adaptive deadline mechanism based on one-way delay (OWD) estimation.

Servers locally measure apparent OWDs between the proxy and each server by comparing send and receive timestamps. These samples are fed into a per-node sliding window maintained by an `Owd` component. For each node, the `Owd` structure maintains a bounded window of recent delay samples (by default 1000 samples) and computes a percentile (e.g., the median or 95th percentile) over that window. The proxy collects these measurements, maintains per-node deadline estimates, and tracks the maximum observed value across servers.

When computing a deadline for a new request, the proxy either uses a static default value or, if sufficient OWD samples are available and adaptive mode is enabled, uses the learned maximum percentile-based estimate. This follows the spirit of Nezha's percentile-based deadline calculation but in a simplified form that is easy to integrate with OmniPaxos-KV. In practice, this adaptive strategy increases the fast-path ratio in stable conditions while still reacting to changes in network conditions and clock quality.

2.6 End-to-End Fast and Slow Path Flows

End-to-end, the system behaves as follows. On the *fast path*, a client submits a request to the proxy, which timestamps and deadlines it, then multicasts it as a DOM message to all servers. Each server inserts the message into its early buffer, and once the deadline is reached, the DOM layer releases it in deadline order. The leader speculatively executes the request against its local key-value state and responds to the proxy, while followers verify the ordering and send fast replies. The proxy aggregates these replies and, if a sufficient number of consistent responses arrive before the fast-path timeout, returns a fast-path response to the client. In parallel, the command is still appended and decided through OmniPaxos so that all replicas converge on the same log even though the client has already received an early reply.

On the *slow path*, there are two entry points. First, the proxy can trigger fallback after a timeout: if a request does not become fast-path committable in time, it marks the request aborted and broadcasts `AbortFastPath`. Second, a request may already be classified as late by DOM and inserted into the late buffer, in which case slow-path processing starts without any fast-path abort. In both cases, durable ordering is performed by the current leader when it drains the late buffer and calls OmniPaxos append for those requests; followers do not append on abort and only clear local pending fast-path state. After OmniPaxos decides the entry, the coordinator emits a `SlowPathReply` to the proxy, which replies to the client only if the request is still pending. If the client already received a fast-path response, the proxy drops the late slow-path reply, preventing duplicate responses while preserving linearizability.

3 Implementation

In this section we describe how the above design is realized within the OmniPaxos-KV codebase and how the main components interact. Our implementation reuses the existing networking and storage infrastructure as much as possible, and introduces new modules for clock simulation, deadline-based ordering, and adaptive deadline computation.

3.1 Clock Simulator and Configuration

The clock simulator is implemented as a reusable `ClockSim` type, parameterized by three values: drift rate, uncertainty bound, and synchronization frequency. These parameters are loaded from configuration files and from a dedicated `ClockConfig` structure. The simulator maintains internal state to track when the last synchronization occurred and how much drift has accumulated since then. Whenever `get_time()` is called, the simulator updates its internal time based on the configured drift rate and potentially triggers a resynchronization if the synchronization interval has elapsed.

To validate the simulator, we provide unit tests that check monotonicity, ensure that synchronization reduces large drift errors, and verify that configuration files are parsed correctly. Additional tests instantiate multiple clocks with different drift parameters and confirm that, after synchronization, their simulated times converge within a small tolerance.

3.2 One-Way Delay Estimation

Adaptive deadline computation relies on measuring and aggregating OWD samples. We implement this in a separate `Owd` module that manages, for each node, a bounded deque of recent delay values. When a new sample is added, the module either pushes it to the back of the deque (if the window is not full) or drops the oldest sample to maintain the window size. To compute an adaptive deadline estimate for a node, the module copies the stored values into a temporary vector, selects the configured percentile using an efficient selection operation, and returns the resulting value. If no samples are available, the module falls back to a configured default deadline.

On the server side, OWD samples are recorded whenever a fast reply is generated. The proxy receives these replies, extracts the per-node deadline lengths, and updates its record of the maximum observed deadline across servers. This maximum value is then used as the adaptive deadline for subsequent client requests.

3.3 Proxy Logic

The proxy component is implemented as an asynchronous task that listens for client messages and server replies over existing network channels. When a client issues an append operation, the proxy constructs a unique key from the client

identifier and command identifier, records the request in its internal state, and queries the clock simulator for the current time. It then chooses a deadline, either the configured default or the current maximum adaptive estimate, and computes a fast-path timeout as a function of the deadline and the expected round-trip time.

The proxy wraps the client message into a DOM message that includes the client identifier, command identifier, payload, send time, and deadline, and multicasts this message to all servers. For each in-flight request, the proxy maintains a reply set that groups fast replies by their content and tracks when enough consistent responses have been observed. A periodic timer checks for fast-path timeouts and triggers fast-path aborts when necessary. In that case, the proxy removes the fast-path reply set, marks the request as aborted, and sends an abort message back to all servers, signalling that the slow path should be used.

3.4 Server Event Loop and DOM Integration

On the server side, we extend the existing OmniPaxos server to integrate the DOM layer and the proxy interactions. Each server runs an event loop that multiplexes several concerns: receiving cluster messages for OmniPaxos, receiving client messages, receiving proxy messages, performing leader election, draining the late buffer, and handling DOM deadlines. The server constructs a `Dom` instance configured with the same clock and OWD parameters as the proxy.

When a DOM message arrives from the proxy, the server inserts it into the appropriate buffer. Periodically, the server queries the DOM layer for the duration until the next deadline and sleeps until that instant. When the sleep elapses, the server calls `handle_deadline()`, which returns the set of messages whose deadlines have just passed. For each such message, the server constructs a command for OmniPaxos and, depending on whether it is currently the leader, either speculatively executes the command and sends a fast reply or simply sends a fast reply and defers durable commitment. OmniPaxos eventually decides the same command sequence, allowing followers and lagging nodes to catch up.

The server also handles slow-path aborts from the proxy. Upon receiving an abort message for a given request, the server removes any optimistic state associated with that request and ensures that the command is appended to the OmniPaxos log in the correct position. Once the decision is committed, the server generates a slow-path reply containing the final result and sends it back to the proxy.

3.5 Client Behaviour

Clients are implemented as lightweight asynchronous processes that send a mix of read and write requests according to a configurable workload pattern. Each client connects either directly to the servers or, in our experiments, via the proxy. Before starting the workload, clients wait for a start signal broadcast

by the servers, ensuring that all nodes begin the experiment in a synchronized fashion.

During the experiment, clients generate requests at configured intervals and with a controlled read/write ratio. Each request is tagged with a monotonically increasing identifier and a key derived from that identifier. For writes, the client stores the value it expects to be written so that responses can be validated. As responses arrive—either directly from servers or via the proxy—the client records their latency and correctness in a `ClientData` structure and eventually persists this information to disk for offline analysis.

4 Testing

Testing focuses on two complementary aspects: validating that the clock simulator and DOM layer behave as intended, and verifying that the overall system remains linearizable under a variety of timing and failure scenarios.

4.1 Clock Simulator Tests

We first validate the clock simulator in isolation using unit tests. These tests instantiate `ClockSim` with different drift rates, uncertainty bounds, and synchronization frequencies, and check several properties:

- **Monotonicity**: consecutive calls to `get_time()` must never go backwards, even when the simulator performs a resynchronization.
- **Drift correction**: for large drift rates combined with periodic synchronization, the effective time difference over a given real-time interval should remain bounded, demonstrating that synchronization corrects accumulated drift.
- **Configuration parsing**: `ClockConfig` instances loaded from configuration files must match expected values, and invalid configurations (e.g., zero synchronization frequency) must be rejected.
- **Convergence**: separate `ClockSim` instances with very different drift rates but the same synchronization frequency are shown to converge to similar times after a synchronization event.

These tests give confidence that the simulated clocks match our intended error model and that their parameters are correctly wired to configuration files.

4.2 DOM and Proxy–Server Integration Tests

To validate the DOM layer and its integration with the proxy and servers, we rely on a combination of targeted tests and end-to-end experiments. At the DOM level, we verify that messages inserted into the early buffer are released in non-decreasing deadline order and that late arrivals are redirected to the

late buffer. We also test that the duration until the next deadline is correctly computed, so that the server event loop wakes up exactly when necessary to process pending requests.

At the proxy and server level, we check that fast-path aborts are triggered correctly when deadlines or fast-path timeouts are exceeded, that abort messages are propagated to all servers, and that slow-path replies are eventually generated once OmniPaxos has committed the corresponding log entries. Logging and metrics are used extensively during development to confirm that requests that are executed on the fast path are not re-applied on the slow path and that the key-value state remains consistent.

4.3 Benchmark Scenarios

For a more realistic evaluation, we run a suite of benchmarks that exercise different clock qualities and fault conditions. Each benchmark is defined by a set of configuration files specifying the clock parameters, server and client behaviour, and proxy settings. All experiments use three servers, two clients, and one proxy. We focus on the following configurations:

- **High quality**: small uncertainty and frequent resynchronization, modelling tightly synchronized clocks with low drift.
- **Medium quality**: moderate uncertainty and synchronization intervals, representing more typical data center conditions.
- **Low quality**: large uncertainty and infrequent synchronization, simulating poor clock synchronization.
- **Adaptive deadline**: the proxy uses OWD-based adaptive deadlines instead of a fixed default, aiming to improve the fast-path ratio while keeping waiting times reasonable. These tests are carried considering the average quality clock configurations
- **Clock skew test**: deliberately introduces clock skew between nodes to stress the DOM layer's assumptions about shared time.
- **Node failure test**: injects server failures with configured probabilities and downtime durations to test robustness under crashes.

For each scenario, we configure the clients to generate a mix of read and write requests over a fixed duration, and we record client-side latencies, per-request outcomes, and server-side metrics such as fast-path and slow-path rates. It's important to notice that we are not checking case with network delays as our fast path deadline is assuring us that if the network is slow and fast replies doesn't arrive, then the fallback is traditional OmniPaxos.

4.4 Correctness and Linearizability

To verify correctness, we collect detailed histories of client operations, including invocation and response times as well as the values returned by reads. These histories are fed into a linearizability checker inspired by the Nezha evaluation harness, which systematically explores whether there exists a serialization consistent with real-time ordering that explains the observed behaviour.

Across all benchmark scenarios considered in this report, the histories are found to be linearizable. In particular, even in the presence of significant clock skew and injected node failures, the system preserves linearizability by falling back to the slow path whenever speculative assumptions about deadlines are violated. This confirms the intended property that clocks are used purely to optimize performance, while safety is always guaranteed by the underlying OmniPaxos log replication.

5 Testing Results

In this section we summarize the performance of our clock-assisted consensus implementation under different clock qualities and fault conditions. For each configuration we report the total number of operations, the mix of puts and gets, end-to-end latency percentiles (p50 and p95), fast-path usage, and average fast-path/slow-path latencies, and we interpret these results in light of our design. We decided to use the same number of requests sent by the client (between 360 and 370) so that we can have good comparisons of what happened between tests. Unfortunately the throughput is dependent on our containers and the connection we are using. We are highly dependent on that so sometimes the servers are a bit faster and sometimes they are not.

Config	Ops	Puts	Gets	p50	p95	Fast-path [%]	Avg fast-path (ms)	Avg slow-path (ms)	Lin.
high_quality	362	180	182	4.26	7.60	83.4	3.56	3.65	PASS
low_quality	363	271	92	5.96	13.63	69.7	4.57	5.58	PASS
medium_quality	362	274	88	3.88	8.14	79.6	3.43	4.49	PASS
clock_skew	362	264	98	4.60	6.46	51.4	4.53	6.57	PASS
node_failure	402	304	98	9.78	2014.19	2.0	5.77	8.49	PASS

Table 1: Summary of benchmark results across all configurations. Latencies are in milliseconds.

5.1 High-Quality Clocks

In the *high-quality* configuration, clocks exhibit low drift and tight uncertainty bounds with frequent resynchronization. The system executes 362 operations, split between 180 puts and 182 gets. Latency is low, with p50 at 4.26 ms and p95 at 7.60 ms. The fast-path ratio reaches 83.4%, and the average fast-path and slow-path latencies are 3.56 ms and 3.65 ms. This matches the hypothesis that better synchronization increases fast-path use and keeps latency low across both paths.

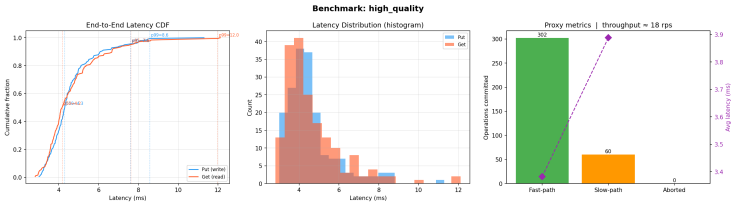

Figure 1: Latency and throughput for the high-quality clock configuration.

5.2 Low-Quality Clocks

The *low-quality* configuration uses clocks with larger uncertainty and less frequent synchronization. The system handles 363 operations (271 puts and 92 gets), with p50 latency at 5.96 ms and p95 at 13.63 ms. The fast-path ratio drops to 69.7%, while average fast-path and slow-path latencies rise to 4.57 ms and 5.58 ms. Compared to the high-quality case, this reflects more frequent fallback to the slow path and higher latency on both paths.

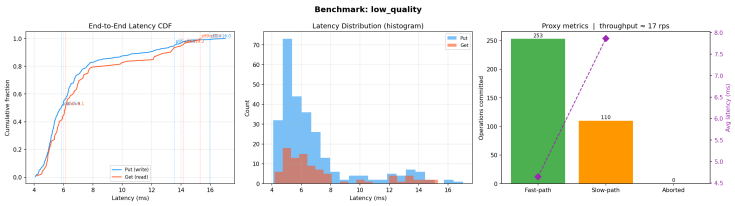

Figure 2: Latency and throughput for the low-quality clock configuration.

5.3 Medium-Quality Clocks

With *medium-quality* clocks, the system processes 362 operations (274 puts and 88 gets). The p50 latency is 3.88 ms and p95 is 8.14 ms, with a fast-path ratio of 79.6%. Average fast-path latency is 3.43 ms and average slow-path latency is 4.49 ms. This configuration provides a strong balance between latency percentiles and path-specific averages.

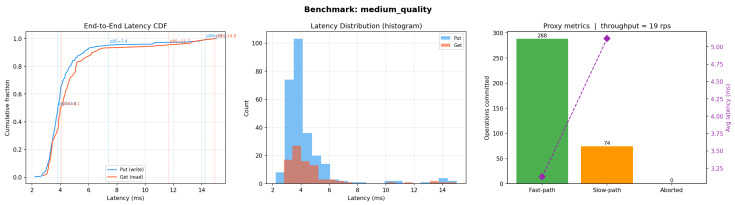

Figure 3: Latency and throughput for the medium-quality clock configuration.

5.4 Clock Skew

The *clock skew* test deliberately introduces skew between node clocks while keeping other parameters comparable. The system processes 362 operations (264 puts and 98 gets), with p50 latency at 4.60 ms and p95 at 6.46 ms. The fast-path ratio decreases to 51.4%, and average latencies increase to 4.53 ms on the fast path and 6.57 ms on the slow path. This shows the expected degradation in fast-path usage under skew while preserving correct fallback behavior.

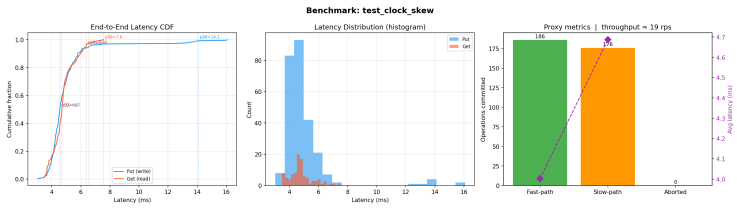

Figure 4: Latency and throughput for the clock skew test configuration.

5.5 Node Failures

Finally, the *node failure* configuration injects server failures with a certain probability and downtime, stressing both DOM and OmniPaxos. The system executes 402 operations, with 304 puts and 98 gets. Latencies are significantly affected: p50 is 9.78 ms and p95 is 2,014.19 ms. The fast-path ratio drops to 2.0%, with average fast-path and slow-path latencies of 5.77 ms and 8.49 ms. Most operations therefore proceed through slow-path handling, but linearizability is preserved.

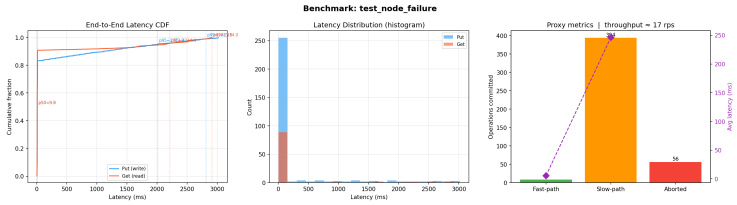

Figure 5: Latency and throughput for the node failure configuration.

5.6 Global Comparison

Taken together, these results support the central hypothesis that clock quality and deadline selection have a strong impact on consensus performance. Configurations with better-synchronized clocks or well-chosen adaptive deadlines achieve higher fast-path ratios and lower p50/p95 latencies, with lower average latencies on both fast and slow paths. As clock quality degrades, or as skew

and failures are introduced, the system falls back more frequently to the slow path and latency metrics worsen, especially p95, but without compromising linearizability. All configurations evaluated in this report passed linearizability checks, reinforcing the design principle that clocks are used solely to optimize performance while correctness is always enforced by the underlying consensus protocol.

6 Adaptive Deadline Evaluation

The adaptive deadline mechanism is implemented in two stages. First, each server estimates one-way delay (OWD) from the proxy by maintaining a sliding window of delay samples in the `Owd` structure. For every fast reply, the server computes a new delay sample and feeds it into the window associated with the proxy node. The `Owd` module then selects a configurable percentile from the window, providing a scalar estimate of the current OWD for that replica. This estimate is exported in the `deadline_length` field of `FastReply` and `SlowPathReply` messages.

Second, the proxy aggregates these per-replica estimates across replies. For each replica, it stores the most recent `deadline_length` in a per-node map and maintains the maximum value over all replicas. When computing a deadline for a new client request, the proxy queries its clock simulator for the current time and uncertainty, and then uses the formula

$$D = send_time + \max_r \widehat{OWD}_{s,r} + \epsilon,$$

where $\widehat{OWD}_{s,r}$ is the most recent percentile estimate from replica r and ϵ is the current uncertainty bound from `ClockSim`. The resulting deadline is attached to the `DomMessage` for that request, and the fast-path timeout is derived from the same adaptive estimate. This design closely follows Nezha's idea of combining per-replica OWD estimates with a global uncertainty term while remaining simple to integrate into the existing OmniPaxos-KV codebase.

Comparing the *medium-quality* and *adaptive deadline* experiments highlights the benefits and limitations of this mechanism. Under medium-quality clocks without adaptation, we obtain a fast-path ratio of 79.6%, p50/p95 latencies of 3.88 ms and 8.14 ms, and average fast/slow-path latencies of 3.43 ms and 4.49 ms. With adaptive deadlines enabled, the fast-path ratio is similar at 77.7%, and p50/p95 latencies are 4.46 ms and 9.71 ms, while average fast/slow-path latencies are 3.59 ms and 4.69 ms. In other words, the simple per-replica percentile estimator we implemented keeps performance close to the medium-quality baseline but does not yield a dramatic improvement in fast-path usage or tail latency. We attribute this to two factors: the relatively small cluster size (three replicas) and the fact that clock quality is already good enough that fixed deadlines work reasonably well. More aggressive percentile choices or larger, more heterogeneous deployments could make adaptive deadlines more beneficial.

Config	Ops	Fast-path [%]	p50 [ms]	p95 [ms]	Avg fast [ms]	Avg slow [ms]	Lin.
medium_quality	362	79.6	3.88	8.14	3.43	4.49	PASS
adaptive_deadline	364	77.7	4.46	9.71	3.59	4.69	PASS

Table 2: Side-by-side comparison of medium-quality clocks with fixed deadlines and adaptive deadlines based on per-replica OWD estimates.

7 Future Work

There are several directions in which this work could be extended. First, our adaptive deadline mechanism uses a relatively simple percentile-based OWD estimator per node. Future work could explore more sophisticated estimators that account for correlations between paths, time-of-day effects, or sudden network changes, and that consider different percentiles depending on workload or service-level objectives. It would also be interesting to validate these mechanisms against real-world clock-synchronization protocols rather than simulated clocks.

Second, the current implementation treats all operations uniformly, regardless of whether they commute. Techniques from Nezha that exploit operation commutativity could allow additional reordering flexibility, increasing the fast-path ratio without endangering correctness. Integrating such optimizations into OmniPaxos-KV would require extending the key-value interface with richer semantics and updating the DOM layer accordingly.

Third, our failure model is limited to crash failures and static cluster membership. Extending the system to handle dynamic reconfiguration, network partitions, and more realistic failure injection patterns would better approximate production environments. Coupling this with richer telemetry and online monitoring of fast- versus slow-path usage could enable automatic tuning of deadline parameters in response to changing conditions.

Another practical avenue is to remove the dedicated proxy hop by moving proxy responsibilities into a lightweight, stateless client-side component. By embedding the DOM-annotating and reply-aggregation logic directly in clients (or in a library they link), we can avoid the extra network round trip between client and proxy after servers reply, potentially improving end-to-end latency. This approach preserves the same correctness guarantees if the client-side component remains stateless (so that durable ordering and replication are still handled by OmniPaxos on the servers) and careful attention is paid to handling retries, duplicate suppression, and slow-path aborts.

8 Summary

This project integrates Nezha-style, clock-assisted consensus ideas into the OmniPaxos-KV key-value store. We implement a configurable clock simulator, a DOM-like deadline-ordered processing layer, adaptive deadline computation based on one-way delays, and proxy and server logic that support both optimistic fast paths and conservative slow paths. Using a suite of benchmarks that vary clock qual-

ity, deadline strategy, and fault conditions, we show that better clock synchronization and well-tuned deadlines substantially increase the fraction of operations that complete on the fast path, thereby improving latency and throughput.

At the same time, the system remains linearizable even when clocks are poor, skewed, or when node failures occur, by safely falling back to the slow path and relying on OmniPaxos log replication for ordering and durability. These results confirm that clock information can be exploited to accelerate consensus without compromising correctness, and they highlight the importance of jointly designing clock synchronization, deadline selection, and consensus mechanisms in modern distributed systems.